Atributos y Métodos

Dpto. Lenguajes y Sistemas Informáticos
Universidad de Granada

Programación y Diseño Orientado a Objetos

(Curso 2024-2025)

Créditos

- Las siguientes imágenes e ilustraciones son libres y se han obtenido de:
  - Emojis, `https://pixabay.com/images/id-2074153/`
- El resto de imágenes e ilustraciones son de creación propia, al igual que los ejemplos de código

Objetivos

- Aprender la utilidad, significado y uso de:
  - Atributos de instancia
  - Atributos de instancia de la clase
  - Atributos de clase
- Aprender a usar métodos de instancia y de clase
- Aprender las diferencias entre Java y Ruby en cuanto a:
  - Atributos de clase
  - Visibilidad privada
- Tomar nota de los errores más frecuentes que soléis cometer
- Usar correctamente las pseudovariables `this` y `self`
- Conocer los especificadores de acceso

Contenidos

1. **Atributos y métodos de instancia**
2. **Atributos y métodos de clase**
   - Ejemplos
3. **Pseudovariables**
4. **Especificadores de acceso. Visibilidad**
   - Ejemplos

Atributos de instancia

- La descripción de las clases incluye los atributos de instancia que tendrá cada objeto que sea instancia de esa clase
- Los atributos de instancia son variables que están asociadas a cada objeto
- Cada instancia tiene su propio espacio de atributos o variables de instancia.
  - Así, cada instancia tendrá los mismos atributos que otra instancia de la misma clase, pero en zonas de memoria distintas
- El **estado** de cada instancia se describe mediante los valores de estos atributos

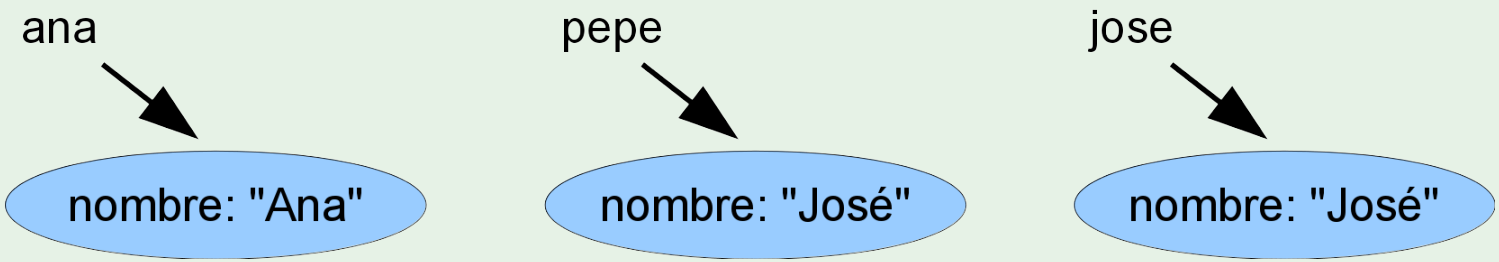
Atributos de instancia

Ejemplo: La clase Persona

```
1  class Persona {
2    private String nombre;
3    // . . .
4  }
```

- Todas las instancias de la clase `Persona` tendrán un atributo denominado `nombre`
  - Existirá una variable denominada `nombre` para cada instancia
  - Dos instancias de `Persona` distintas almacenan el nombre en variables distintas (almacenadas en zonas de memoria distintas) aunque se llamen igual

```
1  Persona ana  = new Persona ("Ana");
2  Persona pepe = new Persona ("José");
3  Persona jose = new Persona ("José");
```



Métodos de instancia

- Son funciones o métodos definidos en una clase y que estarán asociados a los objetos de dicha clase
- Desde los métodos asociados a un determinado objeto son accesibles los atributos de instancia de dicho objeto.
  Tanto para lectura como para escritura.

Ejemplo: La clase Persona

```
1 class Persona {
2   private String nombre;
3   // ...

4   String saludar () {
5     return "Hola, me llamo " + nombre;
6   }

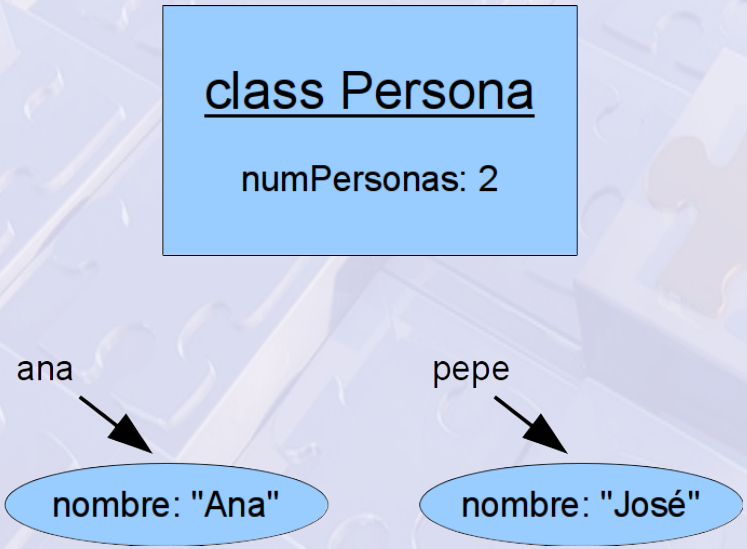
7   void cambiaNombre (String otroNombre)
        {
8     nombre = otroNombre;
9   }
10 }
```

Ejemplo: La clase Persona

```
1 Persona pepe = new Persona ("José")
2 Persona ana = new Persona ("Ana");
3
4 System.out.println (ana.saludar ());
5 // Hola, me llamo Ana
6
7 ana.cambiaNombre ("Ana Belén");
8
9 System.out.println (ana.saludar ());
10 // Hola, me llamo Ana Belén
11
12 System.out.println (pepe.saludar ());
13 // Hola, me llamo José
```

Atributos de clase

- Almacenan información que se considera asociada a la propia clase y **no** a cada instancia
- Son por tanto variables asociadas a la clase y globales a todas las instancias de esa clase
- Cada atributo de clase existe de forma única



Ejemplo: Contador de instancias

- Tanto `ana`, como `pepe` tienen accesible el atributo de clase `numPersonas`
- Cuando su valor cambia, lo hace para todas las instancias
- El atributo es único, está en una zona de memoria asociada a la clase, no a las instancias

Atributos de clase → *Diseño* ←

- ¿Cuándo usarlos?
  - Se debe pensar en ellos cuando se necesite almacenar información que siempre va a ser común a todas las instancias de la clase
  - No tendría sentido que cada instancia de la clase (`Persona` en el ejemplo anterior) guardase una copia del valor almacenado en ese atributo (`numPersonas`)
  - Esto además haría su actualización extremadamente costosa.
- Ejemplos típicos:
  - Contador de instancias
  - Constantes
  - Para evitar el uso de *números mágicos*

    ★ ¿Quién sabría decirme qué es un número mágico?

    Usar números mágicos en los exámenes será penalizado

Métodos de clase

- Son funciones y procedimientos asociados a la propia clase
- Es habitual que accedan/actualicen atributos de clase
- No se puede acceder *directamente* a atributos/métodos de instancia desde un método de clase
  - Sería necesario solicitar ese elemento a una instancia concreta

Java: Contador de instancias

```java
class Persona {
  // atributo y método de clase
  static private int numPersonas = 0;
  static int getNumPersonas () {
    return numPersonas;
  }
  // atributo de instancia
  private String nombre;
  // inicializador
  Persona (String unNombre) {
    nombre = unNombre;
    numPersonas++;
  }
}
```

Ruby: Contador de instancias

```ruby
class Persona
  # atributo y método de clase
  @@num_personas = 0
  def self.num_personas
    @@num_personas
  end
  # inicializador
  def initialize (un_nombre)
    # atributo de instancia
    @nombre = un_nombre
    @@num_personas += 1
  end
end

```

★ ¿Cómo se mostraría el número de instancias creadas?

Atributos de clase: Particularidad de Ruby

- Existen dos tipos de atributos de clase
  - Atributos de clase (`@@atributo_de_clase`)
  - Atributos de instancia de la clase (`@atributo_instancia_clase`)
- Los atributos de clase son accesibles directamente desde el ámbito de instancia.
  - Los atributos de instancia de la clase, no
- Los atributos de clase se comparten con las subclases (herencia). Esto puede ser muy peligroso
  - Los atributos de instancia de la clase, no

Errores frecuentes en Ruby

- Confundir atributos de instancia con atributos de instancia de la clase
  - Hay que tener en cuenta en qué ámbito se está
  - En una clase, cualquier punto dentro de un método de instancia está en ámbito de instancia, lo demás está en ámbito de clase
  - En un ámbito de instacia, `@variable` alude a un atributo de instancia
  - En un ámbito de clase, `@variable` alude a un atributo de instancia de la clase
- Añadir atributos de instancia, atributos de instancia de la clase o atributos de clase cuando hay que usar variables locales
  - Las variables locales y los parámetros de método no llevan @
- Estos errores, en los exámenes, serán penalizados

Ejemplos

Ruby: Confusión entre atributos

```ruby
class Clase
  @@variable = "De clase"
  @variable = "De instancia de la clase"

  def initialize
    @variable = "De instancia"
  end

  def muestraValores
    puts @@variable
    puts @variable
  end

  def self.muestraValores
    puts @@variable
    puts @variable
  end
end

objeto = Clase.new
objeto.muestraValores
Clase.muestraValores
```

★ ¿Cuál es el resultado de ejecutar este programa?

→ *Diseño* ←

- Los nombres de las variables deben ser significativos
- Debe evitarse nombrar a cosas distintas con el mismo nombre

- En el ejemplo anterior  no se han seguido estas recomendaciones por motivos docentes

Ejemplos

Java: Atributos y métodos de clase y de instancia, variables locales

```
1  public class Persona {
2
3    private static final int MAYORIAEDAD=18; //Atributo de clase
4    private LocalDateTime fechaNacimiento;   //Atributo de instancia
5
6    Persona(LocalDateTime fecha) {
7        fechaNacimiento=fecha;
8    }
9
10   public boolean mayorDeEdad() { // Método de instancia
11     LocalDateTime ahora= LocalDateTime.now(); //Llamada a método de clase
12     // "ahora" es una variable local
13
14     //Años completos transcurridos
15     long edad=ChronoUnit.YEARS.between(fechaNacimiento,ahora);
16
17     return (edad>=MAYORIAEDAD);
18   }
19 }
```

★ ¿Qué efecto tiene la palabra `final` en la declaración de `MAYORIAEDAD`?

Ejemplos

Ruby: Atributos y métodos de clase y de instancia, variables locales

```
1  require 'date'
2
3  class Persona
4    @@MAYORIA_EDAD = 18 # Atributo de clase
5
6    def initialize(fecha)
7      @fecha_nacimiento=fecha # Atributo de instancia
8    end
9
10   def mayor_de_edad # Método de instancia
11     ahora = Date.today # "ahora" es una variable local
12     edad = ahora.year - @fecha_nacimiento.year - 1
13     if (ahora.month > @fecha_nacimiento.month)
14       edad += 1
15     else
16       if (ahora.month == @fecha_nacimiento.month)
17         if (ahora.day >= @fecha_nacimiento.day)
18           edad += 1
19         end
20       end
21     end
22     return (edad >= @@MAYORIA_EDAD)
23   end
24 end
```

★ ¿Qué significa la línea 1?

Ejemplos

Ruby: Atributos y métodos de clase y de instancia, variables locales

```
1  class Persona
2      @MAYORIA_EDAD=18 # Atributo de instancia de la clase
3
4      def self.edad_legal # Método de clase (Persona.edad_legal)
5        @MAYORIA_EDAD
6      end
7
8      def initialize(fecha)
9        @fecha_nacimiento = fecha # Atributo de instancia
10     end
11
12     def mayor_de_edad # Método de instancia
13       ahora = Date.today
14       edad = ahora.year - @fecha_nacimiento.year - 1
15       if (ahora.month > @fecha_nacimiento.month)
16         edad += 1
17       else
18         if (ahora.month == @fecha_nacimiento.month) && (ahora.day >= @fecha_nacimiento.day)
19           edad += 1
20         end
21       end
22       return (edad >= self.class.edad_legal) # (Persona.edad_legal)
23     end
24 end
```

★ ¿Se puede prescindir del método `edad_legal`?

★ ¿Cómo quedaría la línea 22?

Ejemplos

Ruby: Atributos y métodos de clase y de instancia, variables locales

```
1  class Producto
2    @@iva = 21 # En mayúscula sería constante
3
4    def initialize(precio, nombre)
5      @precio = precio
6      @nombre = nombre
7    end
8
9    def instanciaSetIVA(iv)
10     # Acceso directo a un atributo de clase desde un método de instancia
11     @@iva = iv
12     # Esto no es posible con atributos de instancia de la clase
13   end
14
15   def self.claseSetIVA(iv)
16     # Acceso directo a un atributo de clase desde un método de clase
17     @@iva = iv
18   end
19
20   def to_s
21     "nombre: #{@nombre}, precio: #{@precio}, iva: #{@@iva}"
22   end
23 end
```

★ ¿Qué diferencia hay entre los métodos de las líneas 9 y 15?

Ejemplos

Ruby: Atributos y métodos de clase y de instancia, variables locales

```
1  # Usando la clase anterior
2
3  p = Producto.new(2, "cosa")
4  puts p.to_s
5
6  p.instanciaSetIVA(25)
7  puts p.to_s
8
9  Producto.claseSetIVA(27)
10 puts p.to_s
11
12 # Lo siguiente no funciona en Ruby
13 # En cualquier caso NO es recomendable
14
15 p.claseSetIVA(50) # esto no funciona en Ruby
16 puts p.to_s
```

★ ¿Qué salida producen las líneas 4, 7 y 10?

Ejemplos

Ruby: Constantes

```ruby
class Test
  @ATRIBUTO_INSTANCIA_CLASE_CTE=1
  @@ATRIBUTO_CLASE_CTE=2
  CTE=3

  def self.modifica
      @ATRIBUTO_INSTANCIA_CLASE_CTE=11 # en realidad no es una constante
      @@ATRIBUTO_CLASE_CTE=22 # en realidad no es una constante

      CTE=33 # dynamic constant assignment. Error por ser constante
      # Este error se notifica aunque no se ejecute este método
  end

  def self.muestra
      puts @ATRIBUTO_INSTANCIA_CLASE_CTE
      puts @@ATRIBUTO_CLASE_CTE
      puts CTE
  end
end

Test.muestra # 1 2 3
Test.modifica
Test.muestra # 11 22 3

# Las constantes no son privadas
puts Test::CTE

Test::CTE=333 #  warning: already initialized constant Test::CTE
```

Pseudovariables

- Existen palabras reservadas que referencian al propio objeto, o a la clase
  - Java: this (también en C++, C#, etc.)
  - Ruby: self (también en Python, Rust, etc.)

Java: this

```
1  class Persona {
2    private String nombre;
3
4    Persona (String nombre) {
5      this.nombre = nombre;
6    }
7
8    Persona () {
9      this ("Anónimo");
10   }
11 }
```

★ Significado de `this` en las líneas 5 y 9

★ Significado de `self` en las líneas 3 y 13

★ Otras formas de escribir la línea 13

Ruby: self

```
1  class Persona
2    @@MAYORIA_EDAD = 18
3    def self.mayoria_edad
4      return @@MAYORIA_EDAD
5    end
6    def initialize (nombre)
7      @nombre = nombre
8    end
9    def nombre
10     return @nombre
11   end
12   def to_s
13     return "Me llamo " + self.nombre
14   end
15 end
```

Especificadores de acceso (Visibilidad)

- Existen distintos niveles de acceso a atributos y métodos
- A este respecto hay diferencias importantes entre lenguajes
- En general:
  - Privado: sin acceso desde otra clase y/o desde otra instancia
  - Paquete: sin restricciones dentro del mismo paquete (no procede en Ruby)
  - Público: sin restricciones de acceso
- Este tema se abordará con detalle en otro módulo

Acceso privado: Diferencias entre Java y Ruby

- **Java**

  se puede acceder a elementos privados (métodos y atributos)

  - Desde una instancia a otra instancia de la misma clase
  - Desde el ámbito de clase a una instancia de esa clase
  - Desde el ámbito de instancia a la clase de la que se es instancia

- **Ruby**
  - Todo lo anterior no está permitido en Ruby
  - Los atributos siempre son privados

Ejemplos de visibilidad

Ruby: Visibilidad

```ruby
class Prueba

  def self.metodoClasePublico
    puts "público de clase"
  end

  private # solo afecta a los métodos de instancia
  def self.metodoClasePrivado # sigue siendo público
    puts "privado de clase"
  end

  def metodoInstanciaPrivado
    puts "privado de instancia"
  end

  # Así también se hace privado el método de instancia
  private :metodoInstanciaPrivado

  # Así se hacen privados los métodos de clase
  private_class_method :metodoClasePrivado
end

Prueba.metodoClasePublico
#Prueba.metodoClasePrivado           # Error, es privado
#Prueba.metodoInstanciaPrivado       # Error, es de instancia
#Prueba.new.metodoInstanciaPrivado   # Error, privado
#Prueba.new.metodoClasePublico       # Error, es de clase
```

Ejemplos de visibilidad

Java: Visibilidad

```
1  public class UnaClase {
2
3    public void metodoPublico() { System.out.println("Público"); }
4
5    private void metodoPrivado() { System.out.println("Privado"); }
6
7    // Todo esto funciona en Java aunque llame la atención
8    public void usoDentroDeClase() {
9      metodoPrivado();
10   }
11
12   public void usoConOtroObjeto() {
13     UnaClase obj2 = new UnaClase();
14     obj2.metodoPrivado();
15   }
16
17   public static void main(String []args) { // Seguimos en UnaClase
18     UnaClase obj = new UnaClase();
19     obj.metodoPublico();
20     obj.metodoPrivado();
21     obj.usoDentroDeClase();
22     obj.usoConOtroObjeto();
23   }
24 }
```

Ejemplos de visibilidad

Ruby: Visibilidad

```ruby
class UnaClase

  def metodoPublico
    puts "Publico"
  end

  def usoDentroDeClase
    metodoPrivado
  end

  def usoConOtroObjeto
    obj2 = UnaClase.new
    # obj2.metodoPrivado   # error, privado de otra instancia
  end

  private
  def metodoPrivado
    puts "Privado"
  end
end

obj = UnaClase.new
obj.metodoPublico
# obj.metodoPrivado    # error, privado
obj.usoDentroDeClase
obj.usoConOtroObjeto
```

Visibilidad → *Diseño* ←

- ¿Qué visibilidad asignar a atributos y métodos?
  - Por regla general, la más restrictiva
  - Privada para los atributos
    - ★ Para los que necesiten ser leídos desde fuera de la clase, se creará un método con visibilidad de paquete o público (según corresponda) que proporcionará dicho atributo *al exterior* (consultor)
    - ★ ¡Cuidado! Si lo que se proporciona es una **referencia**, el atributo podría ser modificado desde fuera de la clase
    - ★ Para los que necesiten ser modificados desde fuera de la clase, se creará un método con la visibilidad adecuada que reciba los parámetros necesarios y, tras realizar las comprobaciones pertinentes, realice la modificación (modificador)
    - ★ Los atributos de un objeto no deberían ser modificados por métodos distintos de los propios de dicho objeto (o de su clase)
    - ★ Solo se crearán los consultores y modificadores necesarios, y con la visibilidad más restrictiva que sea posible

Atributos y Métodos

Dpto. Lenguajes y Sistemas Informáticos
Universidad de Granada

Programación y Diseño Orientado a Objetos

(Curso 2024-2025)